



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4

по дисциплине

«Разработка баз данных»

Выполнил студент группы ИНБО-20-23

Школьный И.В.

Проверил доцент

Исаева М.В.

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА №4. АНАЛИТИЧЕСКИЕ ЗАПРОСЫ: ОКОННЫЕ ФУНКЦИИ И ПОСТРОЕНИЕ СВОДНЫХ ТАБЛИЦ

Цель работы:

Целью данной практической работы является формирование у студентов углубленных навыков работы со сложными аналитическими запросами в СУБД PostgreSQL.

Постановка задачи:

Для выполнения практической работы необходимо последовательно выполнить четыре задачи, используя собственную базу данных. Все примеры в данном документе основаны на демонстрационной базе данных «Аптека», содержащей таблицы `manufacturers` (производители), `medicines` (лекарства) и `sales` (продажи).

Ваша задача — адаптировать каждую из поставленных задач к логической структуре и предметной области вашей базы данных. Приведенные ниже формулировки и последующие примеры кода служат шаблоном для понимания, какой тип аналитического запроса требуется составить.

Задание №1: использование ранжирующих функций

Для каждой основной «родительской» сущности в вашей БД (например, производитель, категория товара, автор) определить три наиболее значимых по некоторому числовому признаку дочерних сущности (например, три самых дорогих товара, три самые популярные книги по количеству продаж).

В результирующей таблице должны быть указаны идентификатор группы, идентификатор дочерней сущности, её числовой признак и ранг. Для расчёта ранга использовать функцию `RANK()` или `DENSE_RANK()`.

Задание №2: использование агрегатных оконных функций

Для ключевой сущности, имеющей транзакции по времени (например, товар, услуга), рассчитать нарастающий итог (кумулятивную сумму) по некоторому показателю (например, объем продаж, количество заказов) с разбивкой по временным периодам (месяцам или годам).

Отчёт должен содержать идентификатор сущности (id/название/...), временной период, сумму за период и кумулятивную сумму.

Задание №3: использование функции смещения

Провести сравнительный анализ общих показателей по периодам.

Для каждого периода (например, месяца), начиная со второго, необходимо вывести общий показатель за текущий период и аналогичный показатель за предыдущий период в одной строке. Это позволит наглядно оценить динамику.

Необходимо использовать функцию LAG().

Задание №4: построение сводной таблицы

Создать сводный отчет, который агрегирует некоторый числовой показатель для основной сущности по категориям, представленным в виде столбцов.

Например, показать общую сумму продаж для каждого товара по кварталам года.

Строки в отчете должны представлять основные сущности, а столбцы — категории. Задачу необходимо решить двумя способами:

1. С использованием условной агрегации (комбинация SUM и CASE).
2. С использованием функции crosstab из расширения tablefunc.

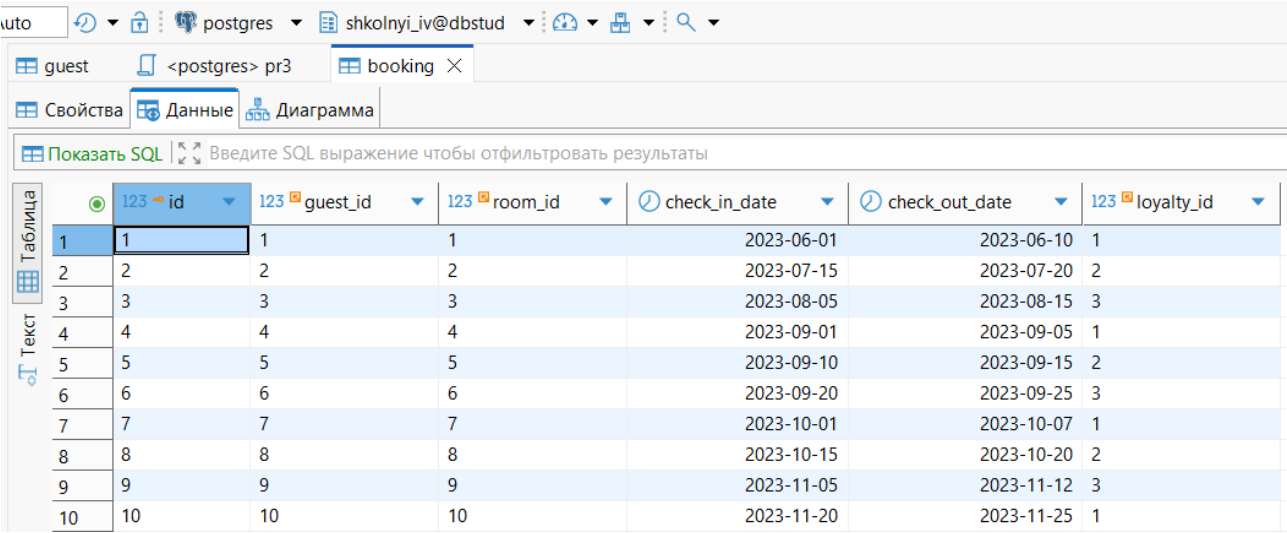
Каждый SQL-запрос сопровождать комментарием, объясняющим его назначение и логику работы с учетом специфики вашей базы данных.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

Введение: от одной таблицы к реляционной модели

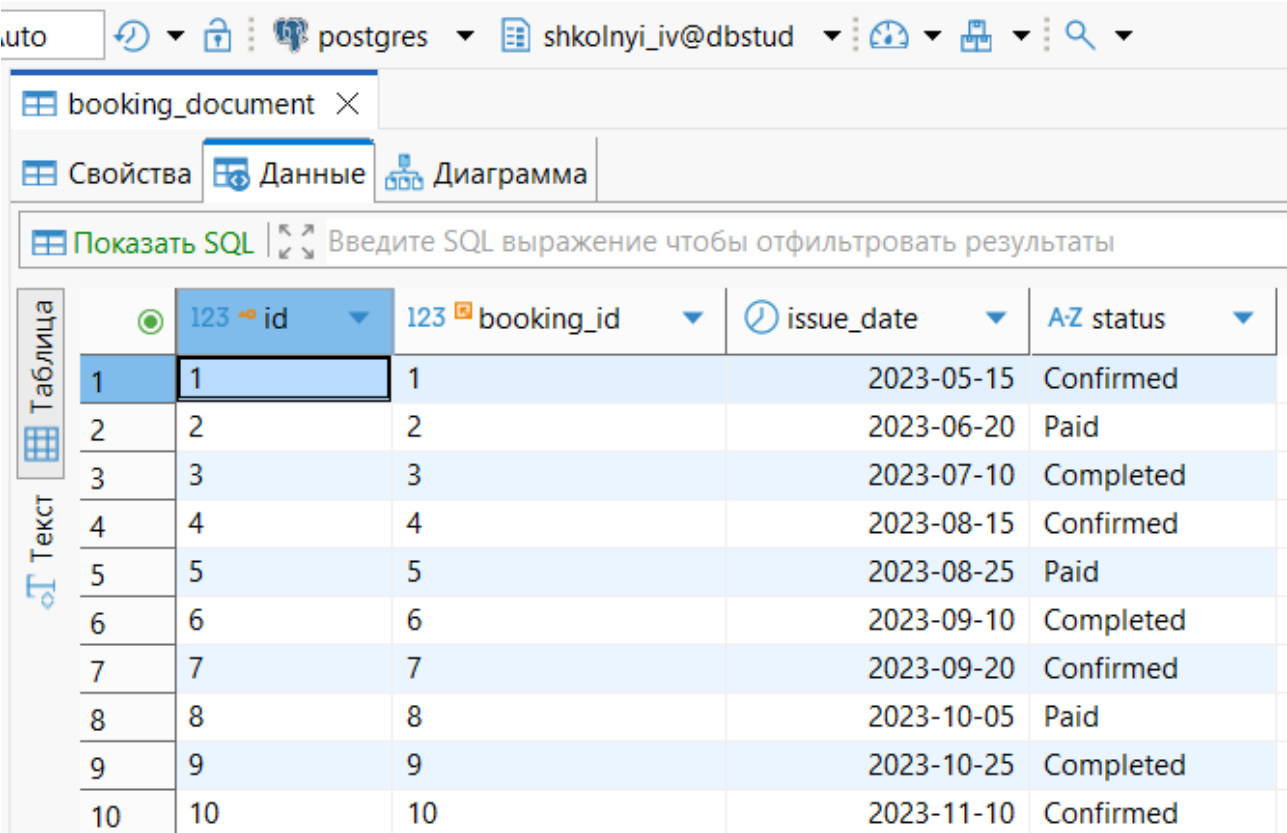
Ниже приводятся таблицы, используемые для построения запросов.

Таблица 1. Таблица Бронирование (Booking)



	123 id	123 guest_id	123 room_id	check_in_date	check_out_date	123 loyalty_id
1	1	1	1	2023-06-01	2023-06-10	1
2	2	2	2	2023-07-15	2023-07-20	2
3	3	3	3	2023-08-05	2023-08-15	3
4	4	4	4	2023-09-01	2023-09-05	1
5	5	5	5	2023-09-10	2023-09-15	2
6	6	6	6	2023-09-20	2023-09-25	3
7	7	7	7	2023-10-01	2023-10-07	1
8	8	8	8	2023-10-15	2023-10-20	2
9	9	9	9	2023-11-05	2023-11-12	3
10	10	10	10	2023-11-20	2023-11-25	1

Таблица 2. Таблица Документы о бронировании (Booking_document)



	123 id	123 booking_id	issue_date	A-Z status
1	1	1	2023-05-15	Confirmed
2	2	2	2023-06-20	Paid
3	3	3	2023-07-10	Completed
4	4	4	2023-08-15	Confirmed
5	5	5	2023-08-25	Paid
6	6	6	2023-09-10	Completed
7	7	7	2023-09-20	Confirmed
8	8	8	2023-10-05	Paid
9	9	9	2023-10-25	Completed
10	10	10	2023-11-10	Confirmed

Таблица 3. Таблица Брошюра (Brochure)

Auto postgres shkolnyi_iv@dbstud

brochure X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 hotel_id	A-Z language	publication_date
1	1	1	Russian	2022-01-10
2	2	2	English	2022-03-15
3	3	3	German	2022-05-20
4	4	1	English	2022-02-28
5	5	2	German	2022-04-10
6	6	3	Russian	2022-06-15
7	7	1	German	2022-07-20
8	8	2	English	2022-08-25
9	9	3	English	2022-09-30
10	10	1	Russian	2022-10-05
11	11	2	Russian	2022-11-12
12	12	3	German	2022-12-18

Таблица 4. Таблица Регистрация (Check_in)

Auto postgres shkolnyi_iv@dbstud

check_in X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 guest_id	123 total_bill	123 room_id	123 doc_id
1	1	1	45 000	1	1
2	2	2	42 500	2	2
3	3	3	120 000	3	3
4	4	4	18 000	4	4
5	5	5	39 000	5	5
6	6	6	57 500	6	6
7	7	7	31 200	7	7
8	8	8	41 000	8	8
9	9	9	84 000	9	9
10	10	10	27 500	10	10

Таблица 5. Таблица Работник (Employee)

uto postgres shkolnyi_iv@dbstud

employee

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	A-Z full_name	123 position_id	hire_date	123 hotel_id
1	1	Смирнова Ольга Дмитриевна	1	2018-06-10	1
2	2	Кузнецов Андрей Петрович	2	2020-02-15	2
3	3	Васильева Елена Ивановна	3	2021-05-20	3
4	4	Петров Максим Игоревич	4	2019-03-12	1
5	5	Семенова Ирина Васильевна	5	2020-08-25	2
6	6	Ковалев Александр Петрович	6	2018-11-30	3
7	7	Новикова Юлия Сергеевна	7	2021-02-14	1
8	8	Данилов Алексей Викторович	8	2017-07-22	2
9	9	Тихонова Марина Олеговна	9	2016-05-18	3
10	10	Фролов Денис Николаевич	10	2022-01-10	1

Таблица 6. Таблица Гость (Guest)

Auto postgres shkolnyi_iv@dbstud

guest

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	A-Z full_name	A-Z phone_number	A-Z email
1	1	Козлова Мария Игоревна	7911123456	kozlova@mail.ru
2	2	Николаев Дмитрий Олегович	7902876543	nikolaev@gmail.com
3	3	Федорова Екатерина Павловна	7922555777	fedorova@yandex.ru
4	4	Громов Сергей Александрович	7913987654	gromov@mail.ru
5	5	Волкова Анастасия Викторовна	7904765432	volkova@gmail.com
6	6	Белов Артем Сергеевич	7923876541	belov@yandex.ru
7	7	Соколова Ольга Николаевна	7915234567	sokolova@mail.ru
8	8	Иванов Иван Иванович	7910123456	ivanov@mail.ru
9	9	Петрова Анна Сергеевна	7909876543	petrova@gmail.com
10	10	Сидоров Алексей Викторович	7921555777	sidorov@yandex.ru

Таблица 7. Таблица Документы гостя (Guest_document)

uto postgres shkolnyi_iv@dbstud

guest_document

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 guest_id	AZ document_data	AZ document_type	issue_date	expiry_date
1	1	1	1234567890	Passport	2015-05-20	2025-05-20
2	2	2	0987654321	Birth certificate	2020-01-15	2030-01-15
3	3	3	1122334455	Driver license	2018-11-30	2028-11-30
4	4	4	4455667788	Passport	2017-03-10	2027-03-10
5	5	5	5566778899	Driver license	2019-08-22	2029-08-22
6	6	6	6677889900	Passport	2016-11-05	2026-11-05
7	7	7	7788990011	Birth certificate	2021-02-14	2031-02-14
8	8	8	8899001122	Driver license	2018-07-30	2028-07-30
9	9	9	9900112233	Passport	2019-12-20	2029-12-20
10	10	10	1011223344	Birth certificate	2020-04-18	2030-04-18

Таблица 8. Таблица Гостиница (Hotel)

uto postgres shkolnyi_iv@dbstud

guest_document hotel

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	AZ name	AZ address	123 star_rating	AZ description
1	1	City Lights	ул. Невский проспект, 45	4	Современный отель в историческом центре
2	2	River Side	наб. реки Мойки, 12	5	Элитный отель с видом на реку
3	3	Park Inn	ул. Садовая, 78	3	Комфортабельный отель рядом с парком

Таблица 9. Таблица Программа лояльности (Loyalty_program)

uto postgres shkolnyi_iv@dbstud

loyalty_program

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	AZ name	123 discount	AZ description
1	1	Gold Member	15	Exclusive discounts and early check-in for gold members
2	2	Silver Member	10	Special offers and room upgrades for silver members
3	3	Bronze Member	5	Basic discounts for frequent guests

Таблица 10. Таблица Должность (Position)

auto postgres shkolnyi_iv@dbstud

loyalty_program position

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать

123 id	A-Z title	123 base_rate
1	Менеджер	50 000
2	Портье	35 000
3	Горничная	30 000
4	Администратор	45 000
5	Шеф-повар	80 000
6	Официант	32 000
7	Охранник	38 000
8	Уборщик	28 000
9	Бухгалтер	55 000
10	Маркетолог	48 000
11	IT-специалист	60 000
12	Директор	100 000
13	Аниматор	35 000

Таблица 11. Таблица Номер (Room)

auto postgres shkolnyi_iv@dbstud

loyalty_program position room

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z room_number	A-Z type	123 price_per_night	123 hotel_id
1	101	Standard	5 000	1
2	202	Deluxe	8 500	2
3	301	Suite	12 000	3
4	102	Standard	4 500	1
5	103	Deluxe	7 500	1
6	201	Standard	4 800	2
7	202	Deluxe	7 800	2
8	203	Suite	11 500	2
9	301	Standard	5 200	3
10	302	Deluxe	8 200	3
11	303	Suite	12 500	3
12	401	Standard	5 500	1

Задание 1. Использование ранжирующих функций

Для каждого отеля определить три самых дорогих номера по цене за ночь.

Листинг 1. Запрос с ранжирующей функцией

```
-- Определение трех самых дорогих номеров в каждом отеле
```

```
WITH ranked_rooms AS (  
  SELECT  
    h.id AS hotel_id,  
    h.name AS hotel_name,  
    r.id AS room_id,  
    r.room_number,  
    r.type,  
    r.price_per_night,  
    DENSE_RANK() OVER (  
      PARTITION BY h.id  
      ORDER BY r.price_per_night DESC  
    ) AS price_rank  
  FROM Hotel h  
  JOIN Room r ON h.id = r.hotel_id  
)  
SELECT  
  hotel_id,  
  hotel_name,  
  room_id,  
  room_number,  
  type,  
  price_per_night,  
  price_rank  
FROM ranked_rooms  
WHERE price_rank <= 3  
ORDER BY hotel_id, price_rank;
```

Этот запрос использует `DENSE_RANK()` для ранжирования номеров в каждом отеле по убыванию цены. Окно разбивается по отелю (`PARTITION BY h.id`), и для каждого отеля выбираются три номера с самой высокой ценой.

Запрос выводит следующий результат (Рисунок 1).

The screenshot shows a PostgreSQL query editor with the following query:

```
WITH ranked_rooms AS ( SELECT h.id AS hotel_id, h.name AS hotel_name, r.id AS room_id, r.number AS room_number, r.type AS type, b.price_per_night AS price_per_night, ROW_NUMBER() OVER (PARTITION BY h.id, r.type ORDER BY b.price_per_night) AS price_rank
```

The result table has the following columns and data:

	123 hotel_id	AZ hotel_name	123 room_id	AZ room_number	AZ type	123 price_per_night	123 price_rank
1	1	City Lights	5	103	Deluxe	7 500	1
2	1	City Lights	12	401	Standard	5 500	2
3	1	City Lights	1	101	Standard	5 000	3
4	2	River Side	8	203	Suite	11 500	1
5	2	River Side	2	202	Deluxe	8 500	2
6	2	River Side	7	202	Deluxe	7 800	3
7	3	Park Inn	11	303	Suite	12 500	1
8	3	Park Inn	3	301	Suite	12 000	2
9	3	Park Inn	10	302	Deluxe	8 200	3

Рисунок 1 — Вывод запроса с использованием ранжирующих функций

Задание 2. Агрегатные оконные функции.

Рассчитать нарастающий итог доходов от бронирований по месяцам.

Листинг 2. Агрегатные оконные функции

```
-- Расчет кумулятивной суммы доходов от бронирований по месяцам
WITH monthly_revenue AS (
  SELECT
    h.id AS hotel_id,
    h.name AS hotel_name,
    DATE_TRUNC('month', b.check_in_date) AS month_period,
    SUM(ci.total_bill) AS monthly_revenue
  FROM Hotel h
  JOIN Room r ON h.id = r.hotel_id
  JOIN Booking b ON r.id = b.room_id
  JOIN Check_in ci ON b.id = ci.room_id AND b.guest_id = ci.guest_id
  GROUP BY h.id, h.name, DATE_TRUNC('month', b.check_in_date)
)
SELECT
  hotel_id,
  hotel_name,
  TO_CHAR(month_period, 'YYYY-MM') AS month,
  monthly_revenue,
  SUM(monthly_revenue) OVER (
    PARTITION BY hotel_id
```

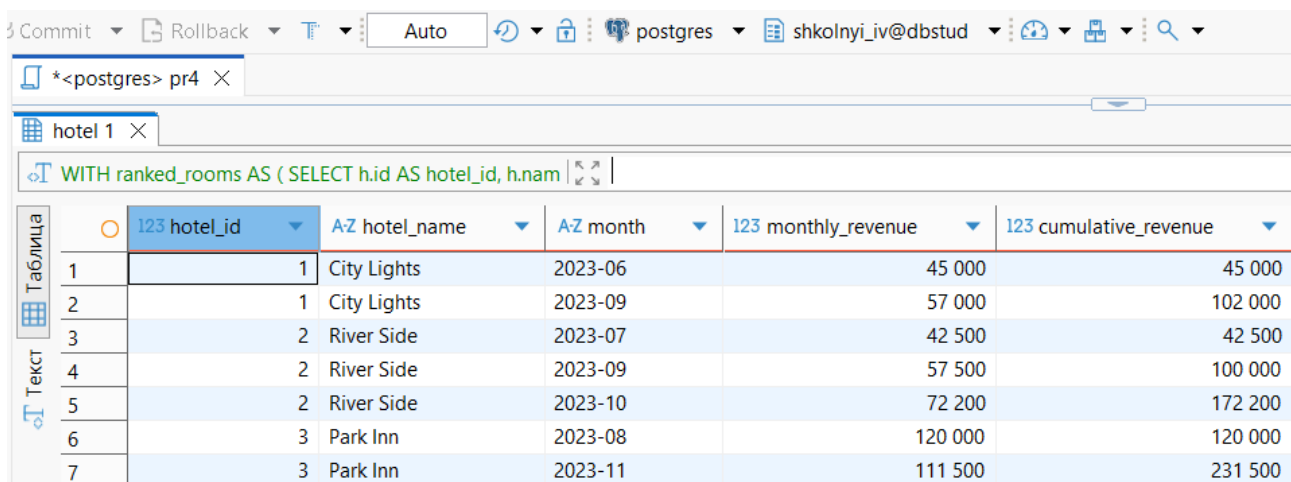
```

ORDER BY month_period
) AS cumulative_revenue
FROM monthly_revenue
ORDER BY hotel_id, month_period;

```

Запрос сначала агрегирует доходы по отелям и месяцам, затем использует оконную функцию SUM() OVER для расчета нарастающего итога для каждого отеля.

Запрос выводит следующий результат (Рисунок 2).



The screenshot shows a PostgreSQL query editor with the following SQL query:

```
WITH ranked_rooms AS ( SELECT h.id AS hotel_id, h.name AS hotel_name, m.month AS month, SUM(r.monthly_revenue) OVER (PARTITION BY h.id ORDER BY m.month) AS cumulative_revenue
FROM hotels h JOIN months m ON h.id = m.hotel_id )
```

The results table is as follows:

	123 hotel_id	A-Z hotel_name	A-Z month	123 monthly_revenue	123 cumulative_revenue
1	1	City Lights	2023-06	45 000	45 000
2	1	City Lights	2023-09	57 000	102 000
3	2	River Side	2023-07	42 500	42 500
4	2	River Side	2023-09	57 500	100 000
5	2	River Side	2023-10	72 200	172 200
6	3	Park Inn	2023-08	120 000	120 000
7	3	Park Inn	2023-11	111 500	231 500

Рисунок 2 — Агрегатные оконные функции

Задание 3. Функция смещения.

Сравнить общие доходы отелей по месяцам с предыдущим периодом.

Листинг 3. Функция смещения

```

-- Сравнение доходов по месяцам
WITH MonthlyRevenue AS (
SELECT
    DATE_TRUNC('month', check_in_date)::DATE AS revenue_month,
    SUM(total_bill) AS total_revenue
FROM
    booking b
JOIN
    check_in ci ON b.id = ci.room_id AND b.guest_id = ci.guest_id
GROUP BY
    revenue_month
)

```

```

)
SELECT
    TO_CHAR(revenue_month, 'YYYY-MM') AS "Месяц",
    total_revenue AS "Доход за текущий месяц",
    LAG(total_revenue, 1, 0) OVER (ORDER BY revenue_month) AS "Доход за предыдущий
    месяц"
FROM
    MonthlyRevenue
ORDER BY
    revenue_month;

```

Этот скрипт:

1. Сначала агрегирует общие доходы по месяцам.
2. Затем использует `LAG(total_revenue, 1, 0)` для получения дохода за предыдущий месяц.
3. Параметр 1 означает "на один период назад".
4. Параметр 0 устанавливает значение по умолчанию для первого месяца (когда предыдущего нет).

Функция `LAG()` используется для получения значения дохода за предыдущий месяц в рамках одного отеля. Также рассчитывается процент роста/падения доходов.

Запрос выводит следующий результат (Рисунок 3).

	AZ Месяц	123 Доход за текущий месяц	123 Доход за предыдущий месяц
1	2023-06	45 000	0
2	2023-07	42 500	45 000
3	2023-08	120 000	42 500
4	2023-09	114 500	120 000
5	2023-10	72 200	114 500
6	2023-11	111 500	72 200

Рисунок 3 — Функция смещения

Задание 4. Построение сводных таблиц

Построим сводную таблицу доходов по отелям и кварталам.

Листинг 4. Условная агрегация

```
-- Сводная таблица доходов по отелям и кварталам (условная агрегация)
SELECT
    h.name AS "Название отеля",
    SUM(CASE WHEN EXTRACT(QUARTER FROM b.check_in_date) = 1 THEN ci.total_bill
    ELSE 0 END) AS "Q1",
    SUM(CASE WHEN EXTRACT(QUARTER FROM b.check_in_date) = 2 THEN ci.total_bill
    ELSE 0 END) AS "Q2",
    SUM(CASE WHEN EXTRACT(QUARTER FROM b.check_in_date) = 3 THEN ci.total_bill
    ELSE 0 END) AS "Q3",
    SUM(CASE WHEN EXTRACT(QUARTER FROM b.check_in_date) = 4 THEN ci.total_bill
    ELSE 0 END) AS "Q4"
FROM
    booking AS b
JOIN
    room AS r ON b.room_id = r.id
JOIN
    hotel AS h ON r.hotel_id = h.id
JOIN
    check_in AS ci ON b.id = ci.room_id AND b.guest_id = ci.guest_id
GROUP BY
    h.name
ORDER BY
    h.name;
```

Этот запрос суммирует общий счет (total_bill) для каждого отеля, разбивая данные по кварталам на основе даты заезда (check_in_date). Каждый CASE проверяет квартал и добавляет сумму только если условие выполняется.

Запрос выводит следующий результат (Рисунок 4).

Commit ▾ Rollback ▾ T ▾ Auto ↺ 🔒 postgres ▾ shkolnyi_iv@dbstud

*<postgres> pr4 ✕

hotel 1 ✕

`SELECT * FROM crosstab( 'SELECT h.name, EXTRACT(QU`

Таблица		A-Z Название отеля ▾	123 Q1 ▾	123 Q2 ▾	123 Q3 ▾	123 Q4 ▾
1		City Lights	0	45 000	57 000	0
2		Park Inn	0	0	120 000	111 500
3		River Side	0	0	100 000	72 200

Рисунок 4 — Условная агрегация

Построим сводную таблицу доходов по отелям и кварталам.

Листинг 5. Метод *crosstab*

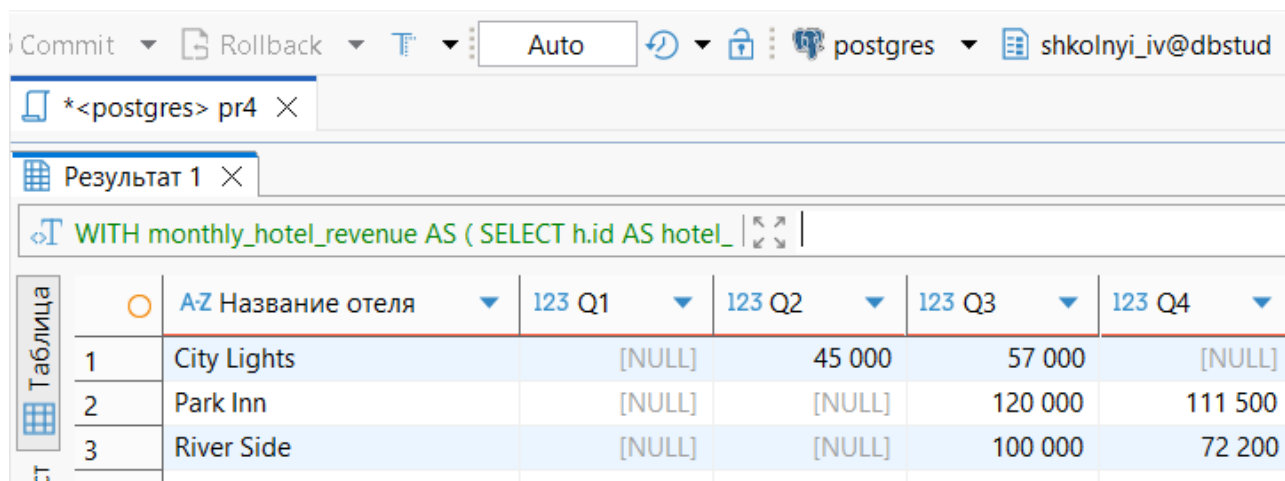
```
-- Сводная таблица доходов по отелям и кварталам (crosstab)
SELECT * FROM crosstab(
  'SELECT
    h.name,
    EXTRACT(QUARTER FROM b.check_in_date),
    SUM(ci.total_bill)
  FROM booking AS b
  JOIN room AS r ON b.room_id = r.id
  JOIN hotel AS h ON r.hotel_id = h.id
  JOIN check_in AS ci ON b.id = ci.room_id AND b.guest_id = ci.guest_id
  GROUP BY h.name, EXTRACT(QUARTER FROM b.check_in_date)
  ORDER BY 1, 2',
  'SELECT q FROM generate_series(1,4) AS q'
) AS ct("Название отеля" TEXT, "Q1" NUMERIC, "Q2" NUMERIC, "Q3" NUMERIC,
"Q4" NUMERIC);
```

Функция *crosstab* принимает:

1. **Основной запрос** - возвращает три колонки: название отеля, квартал, сумму дохода.
2. **Второй запрос** - определяет структуру столбцов (кварталы 1-4).

3. Описание структуры результата - названия и типы колонок в финальной таблице.

Запрос выводит следующий результат (Рисунок 5).



WITH monthly_hotel_revenue AS (SELECT h.id AS hotel_						
Таблица	AZ	Название отеля	123 Q1	123 Q2	123 Q3	123 Q4
1		City Lights	[NULL]	45 000	57 000	[NULL]
2		Park Inn	[NULL]	[NULL]	120 000	111 500
3		River Side	[NULL]	[NULL]	100 000	72 200

Рисунок 5 — Метод crosstab

ВЫВОД

В ходе выполнения практической работы были освоены навыки работы со сложными аналитическими запросами в СУБД PostgreSQL.